17 攤銷分析 解答

17-E01 aggregate table cost

推導

實際插入 16 次。容量 1、2、4、8 滿時擴張，搬移 1、2、4、8 次，共 15 次。總工作為 31，平均 31/16。

結論與檢核

平均 31/16 小於 2，但這只描述整段的計費平均。此序列第 9 次 push 搬移 8 項而昂貴；第 16 次填最後空槽，並未觸發擴張。

17-E02 accounting credit

推導

每次收 3。第 1 次實際 1、容量 1、餘額 2；第 2 次搬 1 加新增 1，實際 2、容量 2、餘額 3；第 3 次搬 2 加新增 1，實際 3、容量 4、餘額 3；第 4 次不擴張，實際 1、餘額 5。擴張發生在超過舊容量的第一筆，而非填滿最後空位的那筆。

結論與檢核

對所有前綴，收費總額不小於實際擴張成本，帳戶不負。常數分配可調整，只要擴張成本能被先前固定費用的盈餘覆蓋。

17-E03 縮表震盪反例

推導

在 size=4 縮成容量 4 後，size 等於容量，下一次插入立即擴回 8，要搬 4 個舊元素。反覆在 4/5 邊界插刪可讓每次操作都接近 Theta(size)。

結論與檢核

可等 size 降到容量 1/4 時才縮半。例如容量 8 在 size=2 時縮成 4，此後插入需回到容量才擴張，中間留下增減緩衝，讓攤銷成本維持 O(1)。完整一般證明須使用本章分段勢能，不能只憑兩三次成功追蹤就斷言所有序列。

17-E04 非零初態的 Counter 勢能

推導

第一次全1→全0，c1=b、Phi0=b、Phi1=0，故c_hat1=b+0−b=0。一般非溢位有t個尾端1，c=t+1、DeltaPhi=1−t，所以c_hat=2；溢位時c_hat=0。相加有sum c_i=sum c_hat_i+Phi0−Phim。

17 攤銷分析 解答 （續）

結論與檢核

因Phim>=0，總成本<=2m+b。不能由本次攤銷成本0宣稱實際成本0，也不能丟掉正的初勢能b而說任意初態總成本<=2m。

17-E05 混合擴縮表的逐案勢能

推導

五次插入操作後(n,M,Phi)依序為(1,1,1),(2,2,2),(3,4,2),(4,4,4),(5,8,2)；c=[1,2,3,1,5]，c_hat=[2,3,3,3,3]。五次刪除後為(4,8,0),(3,8,1),(2,4,0),(1,2,0),(0,0,0)；c=[1,1,3,2,1]，c_hat=[−1,2,2,2,1]。

結論與檢核

實際總成本12+8=20，攤銷總成本14+6=20，兩端Phi都0。單次負攤銷成本表示釋放的勢能多於本次成本，並非負時間。要推廣到任意序列，另需分別證明無擴縮、跨半滿、擴張、收縮、刪空各案的c_hat<=3；本章正文已逐案給出，有限表只核對算式。

17-E06 可成長 Counter 的 Bit 成本

推導

第j位翻轉floor(m/2^j)次，故總翻轉sum_j floor(m/2^j)。令m的binary表示為sum_k b_k2^k，交換有限和得sum_j sum_(k>=j) b_k2^(k−j)=sum_k b_k(2^(k+1)−1)=2m−popcount(m)。因此確切小於2m（m>0）。

結論與檢核

加入DECREMENT後可在2^b−1與2^b間來回；每次跨界都翻b+1位，任意長的來回序列平均Theta(b)，所以不再有與b無關的常數攤銷界。只分析單調遞增的總翻轉，不能自動套到支援反向操作的資料結構。

17 攤銷分析 解答 （續）

原創驗算程式附錄: amortized_checks.py

```
"""Exact operation traces for the stated table and counter cost models."""
from fractions import Fraction
from itertools import product


def potential(n, capacity):
    if capacity == 0:
        assert n == 0
        return Fraction(0)
    return Fraction(2*n-capacity) if 2*n>=capacity else Fraction(capacity,2)-n


def step(n, capacity, operation):
    before=potential(n,capacity)
    copied=0
    if operation=="I":
        if capacity==0:
            capacity=1
        elif n==capacity:
            copied=n
            capacity*=2
        n+=1
    else:
        if n==0:
            return None
        n-=1
        if n==0:
            capacity=0
        elif capacity>=4 and 4*n<=capacity:
            copied=n
            capacity//=2
    actual=1+copied
    after=potential(n,capacity)
    return n,capacity,actual,after,actual+after-before


def trace(operations):
    n=capacity=0
    rows=[]
    for operation in operations:
        result=step(n,capacity,operation)
        if result is None:
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

17 攤銷分析 解答 （續）

```
            return None
        n,capacity,cost,phi,amortized=result
        rows.append((operation,n,capacity,cost,phi,amortized))
        assert phi>=0 and amortized<=3
    assert sum(row[3] for row in rows)==sum(row[5] for row in rows)-potential(n,capacity)
    return rows


def main():
    rows=trace("I"*5+"D"*5)
    assert [row[3] for row in rows]==[1,2,3,1,5,1,1,3,2,1]
    assert [row[5] for row in rows]==[2,3,3,3,3,-1,2,2,2,1]
    assert sum(row[3] for row in rows)==20
    for m in range(1,2049):
        total=sum((m>>j) for j in range(m.bit_length()))
        assert total==2*m-m.bit_count()<2*m
    n=capacity=0
    total=0
    for m in range(1,1025):
        n,capacity,cost,_,_=step(n,capacity,"I")
        total+=cost
        assert total<3*m
    assert sum([1,2,4])==7>5

    legal=0
    for length in range(13):
        for ops in product("ID",repeat=length):
            if trace(ops) is not None:
                legal+=1
    for capacity in (4,8,16,32,64,128):
        for n in range(capacity//4+1,capacity+1):
            for operation in "ID":
                result=step(n,capacity,operation)
                assert result and result[3]>=0 and result[4]<=3
    print(f"PASS table_trace_total=20 table_exhaustive_legal_sequences={legal}
counter_exact_totals=2048 insert_prefixes=1024 amortized_bound=3")


if __name__=="__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study